SAMSUNG INNOVATION CAMPUS

SIC Smart Bank System

A Python Banking Application

How the system thinks: decisions, data flow, and design behind a console-based bank

Console Interface · JSON Persistence · Role-Based Access

Agenda

- **Team**
- Introduction
- Problem Statement
- Methodology
- Results and Insights
- Conclusion
- Q&A

SIC BANK SYSTEM

TEAM 7

- Omar Hamada
- Hassan Badr
- Ahmed Tarek

Agenda

- Team
- **Introduction**
- Problem Statement
- Methodology
- Results and Insights
- Conclusion
- Q&A

INTRODUCTION

What Is the SIC Smart Bank System?

The project is a console-based banking application: everyone interacts with it by typing commands into a terminal, not by clicking a graphical interface.

Its job is to answer one question every time it runs: “who is this person, and what are they allowed to do to their money and their data?”

Every account, transaction, and profile detail the system remembers is kept in a single JSON file, so information survives after the program closes and reloads the next time it starts.

Two kinds of people use it

Regular user: manage their own money — deposit, withdraw, transfer, view history, check ATM status, edit their profile.

Admin: everything a regular user can do, plus system-wide reports, analytics, and the power to update ATM availability.

AT A GLANCE

INTERFACE

Command-line menus (Python input()/print())

STORAGE

A single users.json file on disk

IDENTITY

Numeric account ID, assigned automatically

CORE OPERATIONS

Deposit · Withdraw · Transfer · History

ADMIN-ONLY TOOLS

Reports, analytics, ATM status updates

SHARED TOOLS

ATM lookup, profile editing (view-level differs)

Agenda

- Team
- Introduction
- **Problem Statement**
- Methodology
- Results and Insights
- Conclusion
- Q&A

INTRODUCTION

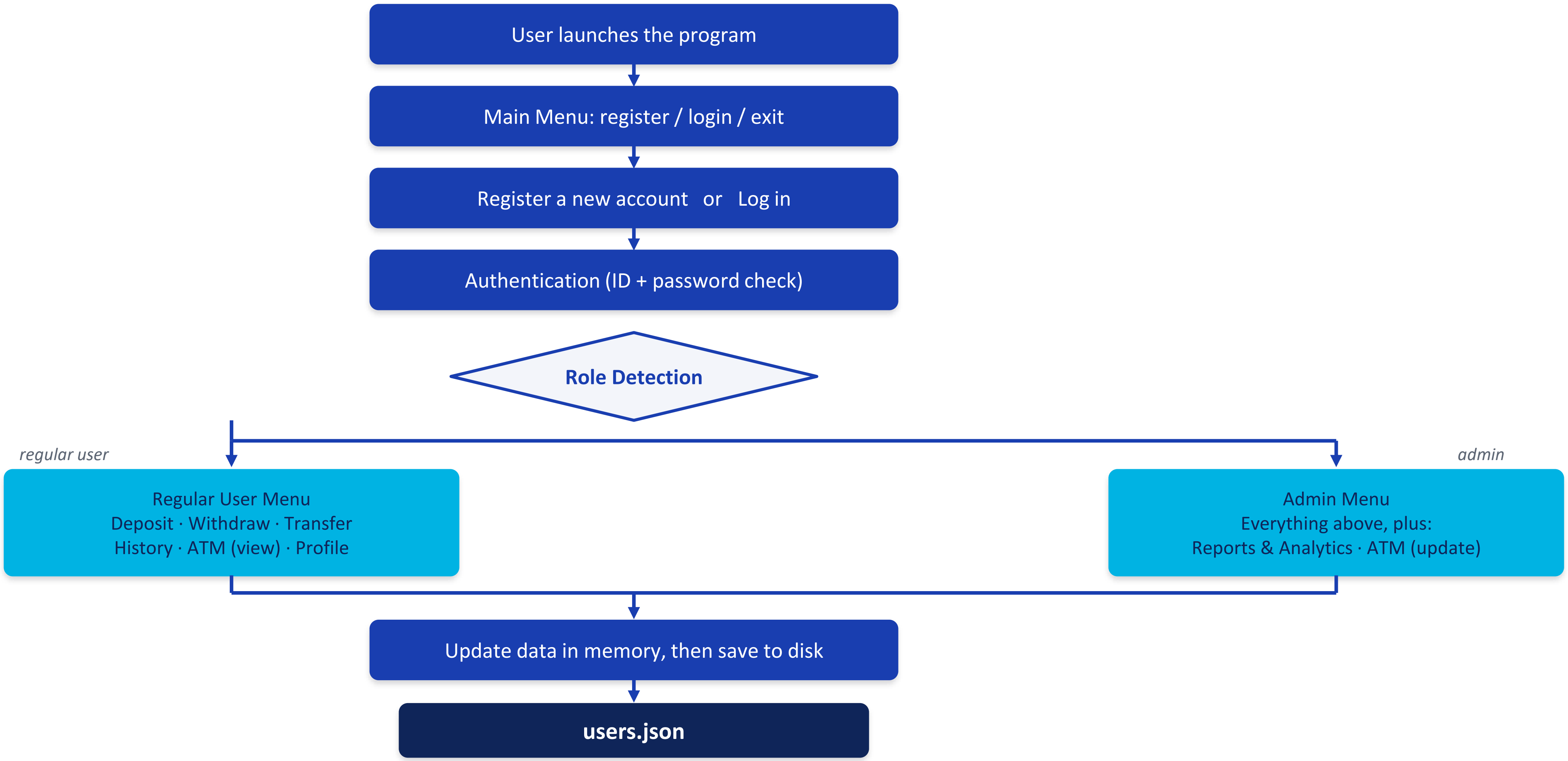
Problem Statement

- **Insecure Account Management:** **Need for structured input validation during sign-up to eliminate duplicate phone numbers and emails.**
- **Data Persistence Risks:** **Requiring immediate and consistent sync between local memory operations and file storage (users.json).**
- **Role Isolation:** **Ensuring standard users cannot access sensitive administrative features like user segment analytics and ATM matrix controls.**

Agenda

System Architecture & Main Flow

Every session follows the same path: prove who you are, then the system decides what menu you get.



ONBOARDING

Registration Logic

1. Collect name, password, phone, email, age, and city
2. **Reject empty fields, duplicate phone/email, or invalid age**
   Any check fails → ask again
3. Generate a new ID (highest existing ID + 1)
4. Create the account: empty wallet, empty history
5. Save the new record to users.json

Why check phone & email uniqueness?

Two accounts should not share the same identifying contact information — phone and email are how a person is recognized outside their account ID, so the system scans every existing user before accepting either one.

Why find-the-max instead of counting users?

New IDs are generated by scanning every account for the highest existing ID and adding one — not by counting how many users exist. That keeps IDs unique even if an account were ever removed.

```
max_id = 0
for u in users:
    if u["id"] > max_id:
        max_id = u["id"]
new_id = max_id + 1
```

AUTHORIZATION

User Roles: Admin vs. Regular User

Authentication answers “who are you?” Authorization answers “what are you allowed to do?” Once the system knows a user’s role, it opens a different menu loop entirely — a simple, direct form of role-based access control (not enterprise-grade security).

CAPABILITY	REGULAR USER	ADMIN
Deposit	✓	✓
Withdraw	✓	✓
Transfer	✓	✓
Transaction history	✓	✓
Reports & analytics (sets, duplicates, frequency)	—	✓
ATM status — view	✓	✓
ATM status — update	—	✓
Update personal profile	✓	✓

The check happens once, right after login: the account’s “stat” field is read, and the rest of the session runs inside either the admin loop or the regular-user loop.

ACCESS CONTROL

Login & Authentication

1. **Enter an account ID → must exist**
   Not found → ask again
2. **Enter password (3 attempts allowed)**
   Wrong 3× → back to main menu
3. Correct password → login succeeds
4. System reads the account's role
5. Admin or regular-user menu is shown accordingly

THE CORE DECISION

Authentication is really one repeated question: "does what the user typed match what's stored?" The system asks it for the ID (does this account exist?) and then for the password (up to three times), so a wrong guess doesn't lock a legitimate user out immediately, but also can't be retried forever in one session.

After login: role decides the menu

Every account carries a "stat" field of either "admin" or "user." The system checks it once, right after a successful login, and branches into one of two separate menu loops.

```
if password == stored_password:
    login_success = True
```

DATA DESIGN

Data Structure & JSON Storage

The system needs to remember many users, and each user needs an identity, personal details, money, and a record of activity. That requirement shapes the data model: one dictionary per user, holding nested groups for each concern.

One user record

- **Identity:** id (auto-generated)
- **Profile:** name · password · phone · email · gender · age · city · account type
- **Wallet:** balance · currency
- **History:** a growing list of transaction strings

Why JSON?

The program keeps every user in a Python list of dictionaries while it runs — but memory disappears when the program closes. The system reads users.json once at startup, and re-writes the whole file after every change that must persist.

```
with open("users.json") as f:
    users = json.load(f)

def save_users():
    json.dump(users, f, indent=4)
```

Deposit Logic

1. Enter an amount and a currency (EGP / SAR / USD)
2. **Validate amount > 0 and currency supported**
   Invalid → reject
3. Convert to EGP using the project's fixed rate
4. Add to balance, record it, save to disk

Fixed conversion rates currently implemented

EGP used directly, no conversion

SAR × 9 → EGP

USD × 30 → EGP

These are constants written into the program — not real-time market exchange rates. A live system would pull rates from an external source.

```
if currency == "SAR":
    amount_in_egp = value * 9
elif currency == "USD":
    amount_in_egp = value * 30
else:
    amount_in_egp = value
```

Withdraw Logic

1. User requests a withdrawal amount
2. **Validate: positive, and ≤ current balance**
   Fails → reject, balance unchanged
3. Deduct from balance, record it
4. Save to disk, confirm success

THE ONE RULE THAT MATTERS

A withdrawal can never push the balance below zero. The system compares the requested amount against the stored balance before touching it — if the check fails, nothing about the account changes.

Insufficient → Reject

Sufficient → Deduct

```
if withdraw_value > balance:
    print("not enough money")
else:
    balance -= withdraw_value
    history.append(msg)
```

Transfer Logic

1. Sender enters the target user's ID
2. **Does that ID exist?** — NO → Reject: invalid ID
3. YES — **Is the target the sender?** — YES → Reject: can't transfer to self
4. NO — **Is sender's balance enough?** — NO → Reject: insufficient funds
5. YES — Subtract amount from sender's balance
6. Add amount to receiver's balance
7. Record the transfer in both histories
8. Save updated data to users.json
9. **Transfer successful**

```
sender["balance"] -= amount
receiver["balance"] += amount

sender["hestory"].append(
    f"transfer to {name}")
receiver["hestory"].append(
    f"transfer from {name}")
```

Transaction History

A bank account is meaningless without a record of what happened to it. Every time a deposit, withdrawal, or transfer succeeds, the system appends a short description to that user's personal history list — nothing is logged until the operation actually succeeds.

Deposit

"Deposit: 500 EGP"

Withdraw

"Withdraw: 200"

Transfer (sent)

"transfer to Sara: 150"

Transfer (received)

"transfer from Ahmed: 150"

Reading history back

When a user asks to see their history, the system does not analyze anything — it simply walks through the stored list, in order, and prints each entry. The value is in having kept the record in the first place, not in the loop itself.

```
if user["hestory"] == []:
    print("no history yet")
else:
    for entry in user["hestory"]:
        print(entry)
```

This same list of raw strings is what the admin's Reports menu later parses to build analytics — see the next slides.

Agenda

ADMIN TOOLS

Reports & Analytics

Once a bank has many users, someone needs to make sense of them as a group, not one account at a time. The admin's Reports menu exists to answer questions like these:

?

Who's active?

Which accounts are flagged active right now

?

Who's VIP?

Which accounts hold VIP status

?

Who struggled to log in?

Accounts with recorded failed login attempts

?

Who transferred money?

Accounts whose history includes a transfer

Four tools on the Reports menu

- Duplicate detection — repeated phone numbers or emails across accounts
- User set analysis — union / intersection / difference across the segments above
- Transaction reports — how often each transaction type happens
- Membership check — is one specific user in a given segment?

Why sets? Python's set type makes "is this person in this group?" and "who's in both groups?" fast and simple to express — exactly the operations this menu needs.

ADMIN TOOLS

Set Operations, Worked Example

Take two segments the system tracks — Active users and VIP users — and ask four different real-world questions about them.

ACTIVE

`{ Ahmed, Ali, Omar }`

VIP

`{ Ali, Omar, Sara }`

UNION	INTERSECTION	DIFFERENCE	SYMMETRIC DIFFERENCE
Active OR VIP	*Active AND VIP*	*Active but NOT VIP*	*In one group, not both*
{ Ahmed, Ali, Omar, Sara }	**{ Ali, Omar }**	**{ Ahmed }**	**{ Ahmed, Sara }**
Everyone in either group	People in both groups at once	Active people who aren't also VIP	Exactly one membership, never both

The point isn't the Python method names — it's that each operation answers a distinct, real analytical question about the user base.

Duplicate Detection & Transaction Analytics

Duplicate Detection

1. Collect every phone number / email into a list
2. Track “seen” values with a set as it scans
3. A value seen twice → mark it duplicate

```
if phone in seen_phones:
    duplicate_phones.add(phone)
else:
    seen_phones.add(phone)
```

The goal: catch accounts that share contact details — a signal worth investigating, even though registration already blocks exact duplicates going forward.

Transaction Frequency Analysis

1. Scan every user’s transaction history
2. Identify each entry’s type, count occurrences
3. Flag types repeated more than once per user

```
type_ = entry.split(":")[0]
freq[type_] = freq.get(type_, 0) + 1
```

This turns a pile of free-text history strings into a simple count — the closest thing this project has to real reporting.

INFRASTRUCTURE

ATM / Branch Management

ATM availability across branches is represented as a grid — a 2D list where each row is a branch and each column is a machine at that branch. 1 means available, 0 means out of service.

	C1	C2	C3	C4
Row 1	1	1	0	1
Row 2	1	0	1	1
Row 3	0	1	1	0
Row 4	1	1	1	0

1 = Available

0 = Out of Service

1. Display the grid, count available vs. out-of-service
2. **Admin picks a row/column and a new status**
   Out of range → reject
3. Update that cell, show the refreshed grid

VIEW VS. UPDATE

Both roles can display the grid and count machines. Only admins can change a machine's status — the update option simply isn't offered on the regular-user menu.

ACCOUNT MAINTENANCE

Profile Management

People change phone numbers, move cities, and forget passwords — so both admins and regular users can edit their own profile.

City	Phone number	Password	Emergency contact (add)	Any optional field (remove)

1. Choose which field to change
2. Enter the new value
3. Profile is updated and saved to users.json

A SNAPSHOT IS TAKEN FIRST

Before any change, the system copies the current profile with copy.deepcopy() and stores it as a “snapshot.”

As implemented today, that snapshot is only stored — there is no “undo” option yet to restore it. A true rollback feature would be a natural next step.

UNDER THE HOOD

Python Concepts Demonstrated

Every concept below was used as a tool to build the system's logic — not as an exercise in syntax.

Lists	Dictionaries	Sets	Loops	Conditions
The users list, and each user's transaction history	User records, profiles, and wallets — nested key/value data	Reports: duplicate detection and segment analysis	Searching by ID/phone/email, validating input, printing history	Every decision the system makes, from login to transfers
JSON	**Functions**	**2D Lists**	**copy.deepcopy()**	**update() / pop()**
Persistent storage across program runs	Reusable logic such as save_users()	The ATM availability matrix	Snapshotting a profile before it's edited	Adding and removing profile fields

HONEST ASSESSMENT

Current Limitations & Future Improvements

CURRENT LIMITATIONS	FUTURE IMPROVEMENTS
• Passwords stored & compared in plain text, no hashing	**Security:** Password hashing, masked input, account lockout
• No database — one flat JSON file, no concurrency control	**Database:** Move to SQLite/PostgreSQL for consistency & concurrency
• Admin & user menus repeat nearly identical code	**Architecture:** Adopt OOP, separate UI / logic / data layers
• Exchange rates are hard-coded, not live	**Banking:** Configurable rates, transaction IDs & timestamps
• History is free-text, not structured (no IDs or timestamps)	**Testing:** Unit and integration tests
• Segment flags (active/VIP) aren't set anywhere yet	**UX:** A GUI or web interface

Together for Tomorrow!
Enabling People
Education for Future Generations